A Neuro-Symbolic Visual Ensemble with VLM Guidance and Visual Routine Composition

Wangcheng Xu WANGCHENG.XU@NORTHWESTERN.EDU
Kenneth D. Forbus FORBUS@NORTHWESTERN.EDU
Qualitative Reasoning Group, Northwestern University, Evanston, IL 60208 USA

Abstract

Understanding diagrams requires fine-grained spatial and relational perception and intermediate representations that can be inspected, corrected, and reused for systems to collaborate with people. End-to-end Vision-Language Models (VLMs) answer diagram questions directly, but opaquely. Without explicit intermediate structure connecting an answer to the image evidence that supports it, an error is difficult to localize and a correct reading is difficult to build upon. Prior work on the Heuristic-based Visual Ensemble (HVE) produced explicit, human-readable intermediate representations with an ensemble of off-the-shelf vision components, but used rigid, per-genre pipelines. We present the Neuro-symbolic Visual Ensemble (NVE), a more dynamic and agentic design in which the VLM guides perception based on visual routines: it interprets each image on the fly, composes a tailored chain of generic visual routines from a shared closed library, and examines and refines their outputs where renderings are available. Spatial structures are computed by the visual routines, with entity geometry providing image grounding. A single mechanism, with no genre-specific perceptual pipelines, is evaluated on 2,220 items across four evaluation sets spanning FoodWebs, BlindTest, BlindTest Extended, and VGRP puzzle boards. Against direct prompting of the same Qwen3.6 backbone, NVE performs better on BlindTest Extended and VGRP cell-level perception, but worse on FoodWebs and BlindTest. Although direct Gemini 3.7 Flash is stronger across all four evaluations, NVE remains training-free and produces explicit, image-grounded intermediate representations designed to be compatible with CogSketch and reusable by downstream reasoning systems.

1. Introduction

Diagrams communicate structure: which organism feeds on which, how stations are connected, what value occupies a cell. A long-standing challenge is for machines to extract and interpret this conceptual and structured information from raw images. Vision-Language Models (VLMs) provide impressive out-of-the-box capabilities for many vision tasks, and their reading of fine-grained structural evidence, such as thin and crossing lines, arrowheads, small marks, and dense grids, varies widely across models and diagram genres (Rahmanzadehgervi et al., 2024; Ren et al., 2025). But whatever a given model's accuracy, an end-to-end answer alone provides no explicit intermediate structure connecting it to the image evidence that supports it. Without such structure, an error is difficult to localize to the line, cell, or arrowhead that caused it. A correct answer alone yields no reusable representation for downstream reasoning. There is no structural artifact a person

© 20XX Cognitive Systems Foundation. All rights reserved.

or program can inspect, correct, or build upon. Reasoning models that produce textual reasoning traces can provide an account of their stated solution strategy. This account offers a reference for understanding the response, but does not by itself provide an image-grounded explanation or an explicit visual intermediate representation, nor reveal the internal mechanism that produced the answer. For diagram understanding as a component of larger reasoning systems, grounded and inspectable intermediate representations make visual interpretations easier to understand and revise, while providing explicit relational structure that can support subsequent symbolic reasoning.

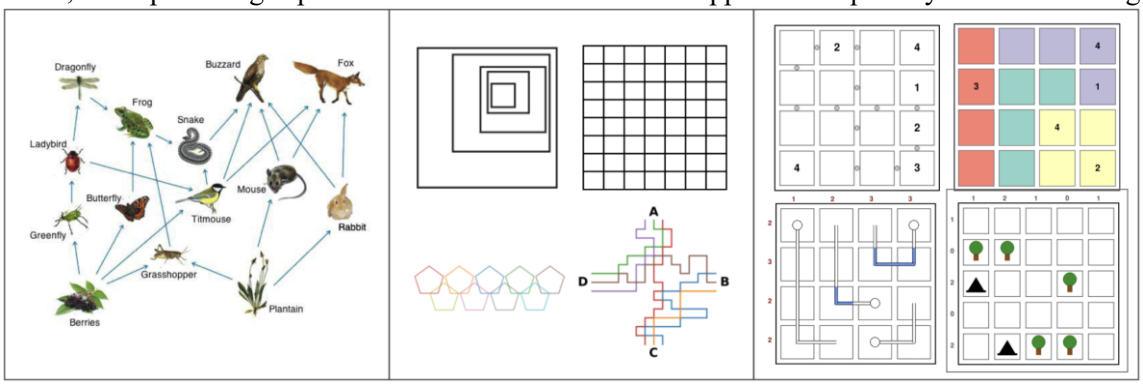


Figure 1. Selected examples from the evaluated visual domains. Left: a FoodWebs diagram. Center: examples from several BlindTest task families. Right: examples from several VGRP-Bench puzzle families.

Our prior work on the Heuristic-based Visual Ensemble (HVE) (Xu & Forbus, 2025) demonstrated an alternative: an ensemble of off-the-shelf vision components producing explicit, grounded, human-readable intermediate representations, with relation extraction on top. HVE's lesson was double-edged. Explicit intermediate representations work but HVE's pipelines were fixed per-genre: each diagram family needed hand-engineered steps, thresholds, and heuristics, and within the perceptual pipeline, the VLM had a fixed role in filtering and classifying text labels.

This paper presents the Neuro-symbolic Visual Ensemble (NVE), which inverts the division of labor. In NVE the VLM guides: it first proposes an open, per-image schema, a task-conditioned specification of the types of visual elements and salient relations to recover and the required answer form. It then composes a chain, an ordered sequence of visual routines from a closed, generic library, to construct each schema item, examines rendered intermediate results and requests refinement, and makes bounded choices among alternatives derived from task declarations or measured image evidence. The visual routines perceive: entities, relations, and sites are constructed by inspectable operations over the image, with entity geometry providing grounding to image evidence. Final answer adaptation is mechanically constrained to bounded transformations of extracted structure.

The contributions of this paper are:

1. We built NVE, a training-free architecture in which a VLM guides while a closed library of general-purpose visual routines constructs visual structure, with no branches specific to

diagrams of a particular genre or theme. The resulting relational representation grounds entities in image evidence and records relations and task answers.
2. We evaluate NVE on 2,220 items across four evaluation sets: 490 food-web diagrams (Krishnamurthy, Tafjord, & Kembhavi, 2016), a fixed 470-item stratified BlindTest sample (Rahmanzadehgervi et al., 2024), 120 generated items in BlindTest Extended, and 1,140 VGRP-Bench grid-puzzle boards involving colored regions, symbols, numbers, and local spatial constraints (Ren et al., 2025). We compare NVE with direct prompting of the same Qwen3.6 backbone and with direct Gemini 3.7 Flash.

The remainder of the paper is organized as follows. We first situate NVE in the staged-vision and sketch-understanding lineage it inherits and the recent visual-programming line it differs from (Section 2). We then describe the architecture: schema construction, the routine library, composition and validation, the examine-refine loop, and projection and closure-gated answer adaptation (Section 3). Section 4 reports the main evaluation and analyzes reuse and error attribution. Section 5 reports paired architectural ablations. We close with limitations and conclusions.

2. Background and Related Work

2.1 Staged Vision and Visual Routines

Human visual understanding is staged. Early vision constructs primitive structure: edges, bars, and boundaries (Marr, 1982). Objects are understood through parts and the relations among them (Biederman, 1987). Many relational properties are extracted not in a single glance but by visual routines, sequential and composable processes such as tracing, indexing, and marking (Ullman, 1984). Qualitative spatial representations make the results of such processes available to reasoning without metric coordinates (Cohn & Renz, 2008). NVE takes this lineage literally: its library is organized as composable routines over progressively richer structure.

2.2 CogSketch and the Missing Front End

CogSketch (Forbus et al., 2011) demonstrated that explicit visual representations, including glyphs with inspectable geometry, qualitative relations over them, and analogical comparison, support visual problem solving, visual relation detection, and sketch recognition (Chen & Forbus, 2021; Chen et al., 2023; Forbus & Lovett, 2021). What CogSketch lacks is a general front end from raw schematic images to glyph-level structure. The Hybrid Primal Sketch (Forbus et al., 2024) and HVE (Xu & Forbus, 2025) are steps toward that front end. NVE extends this line with outputs designed for CogSketch compatibility, including grounded entities and explicit relations (Appendix A).

2.3 HVE: What Transfers and What Does Not

HVE combined text detection, segmentation, line extraction, and geometric heuristics into per-genre pipelines for bar charts and food webs, with a VLM filtering and classifying chart labels. Three lessons transfer to NVE. Explicit grounded intermediates help structural extraction. Training-

free relation parsing is practical. Ensemble outputs improve VLM recall beyond OCR-only augmentation. One lesson motivates NVE's redesign: each genre required hand-engineered steps and thresholds, so generality demanded moving pipeline construction itself into the system. NVE implements this shift through a per-image schema, composition from a shared routine catalog, and examination of outputs where renderings are available.

2.4 VLM Tool Use and Visual Programming

Several lines of VLM tool use motivate NVE. VISPROG and ViperGPT generate modular visual programs over pretrained components, but execute the generated program without NVE's schema-conditioned validation and examine-refine loop (Gupta & Kembhavi, 2023; Surís, Menon, & Vondrick, 2023). Tool-using agents such as MM-REACT and Chameleon orchestrate broad tool pools, whereas NVE restricts execution to a closed typed routine catalog whose products persist as image-grounded structure (Lu et al., 2023; Yang et al., 2023b). Set-of-Mark motivates numbered candidate renderings for bounded selection (Yang et al., 2023a), while Visual Sketchpad motivates rendered visual scaffolding (Hu et al., 2024). NVE uses such renderings to inspect routine outputs. GENOME's reusable modules motivate NVE's non-executable chain sketches, worked examples of routine compositions used as advisory routing hints, while NVE keeps executable capability fixed (Chen et al., 2024). Across these comparisons, the VLM may select, arrange, and judge, while perceptual computation remains grounded in closed-vocabulary routines.

2.5 Benchmarks

We evaluate four sets totaling 2,220 items. An item is an image-task instance with its reference answer. FoodWebs (Krishnamurthy et al., 2016) contributes 490 diagrams with gold consumption relations. BlindTest (Rahmanzadehgervi et al., 2024) contributes a fixed 470-item stratified sample across seven task families: nested shapes, Olympic-style shape counting, line intersections, touching circles, circled letters, subway path counting, and counting grids. Strong direct models are near saturation on this public benchmark, so we also evaluate BlindTest Extended, a companion of 120 new items generated by the authors following six BlindTest task designs (20 per family). We extend the upper scale of each task design, with ranges (original → Extended) of 15-20 → 11-43 characters for circled-letter identification, 3-10 → 4-19 rows and 3-10 → 2-24 columns for grid counting, 0-2 → 2-9 for line intersections, 2-5 → 2-29 for nested squares, 5-9 → 5-29 for Olympic-style shape counting, and 0-3 → 0-5 paths per station pair for subway connectivity. The two suites are not paired item-for-item. BlindTest Extended also asks for a complete pair table on subway items rather than BlindTest's single queried pair. Cross-suite comparisons are therefore robustness trends rather than controlled difficulty curves. VGRP-Bench (Ren et al., 2025) contributes 1,140 grid-puzzle boards that require reconstructing structured grids with symbols, colors, numeric clues, and local spatial constraints.

3. The Neuro-symbolic Visual Ensemble

3.1 Overview and the Governing Principle

Given an image and a task description, NVE produces typed entities with pixel groundings such as boxes, masks, and polylines, together with relations, task answers, and routine provenance. Figure 2 shows the overall workflow, and Algorithm 1 summarizes its execution. Stage 1 constructs a per-image schema: a task-conditioned specification of what visual structure should be recovered (element and relation types, not detected instances), the required answer shape, and closed advisory fields such as rendering class, answer value type, and direction convention. A food-web schema, for example, can specify organism and connector types, a directed consumption relation, a pair-list answer, rendering class, and direction convention. Stage 2 processes schema items in order: elements, structural relations, then task outputs. Projection then assembles the grounded representation into the required answer form, with shape-appropriate, bounded adaptation during pair projection or after grid projection. Appendix A follows two examples through schema construction, routine composition, execution, rendered examination, and answer projection.

HVE showed the value of explicit grounded intermediates, but its per-genre pipelines kept composition with the engineer. NVE shifts that flexibility to three VLM guidance roles: a schema constructor specifies what to recover, a composer assembles routine chains, and an examiner judges rendered results. These roles guide perception based on visual routines and require no NVE-specific training. Section 5.3 tests guide exchangeability.

The governing principle is that visual routines construct perceptual structure. Detection, localization, grouping, connection, and counting are computed by routines, with entity geometry providing image grounding. VLM outputs enter content only through bounded labeling, selection among available candidates, and symbol reading at routine-localized sites. Task-level declarations come from the task text, while image-specific alternatives can be derived from routine measurements. Semantic specification remains open-vocabulary, but perceptual construction stays within the fixed routine library and grounded image evidence.

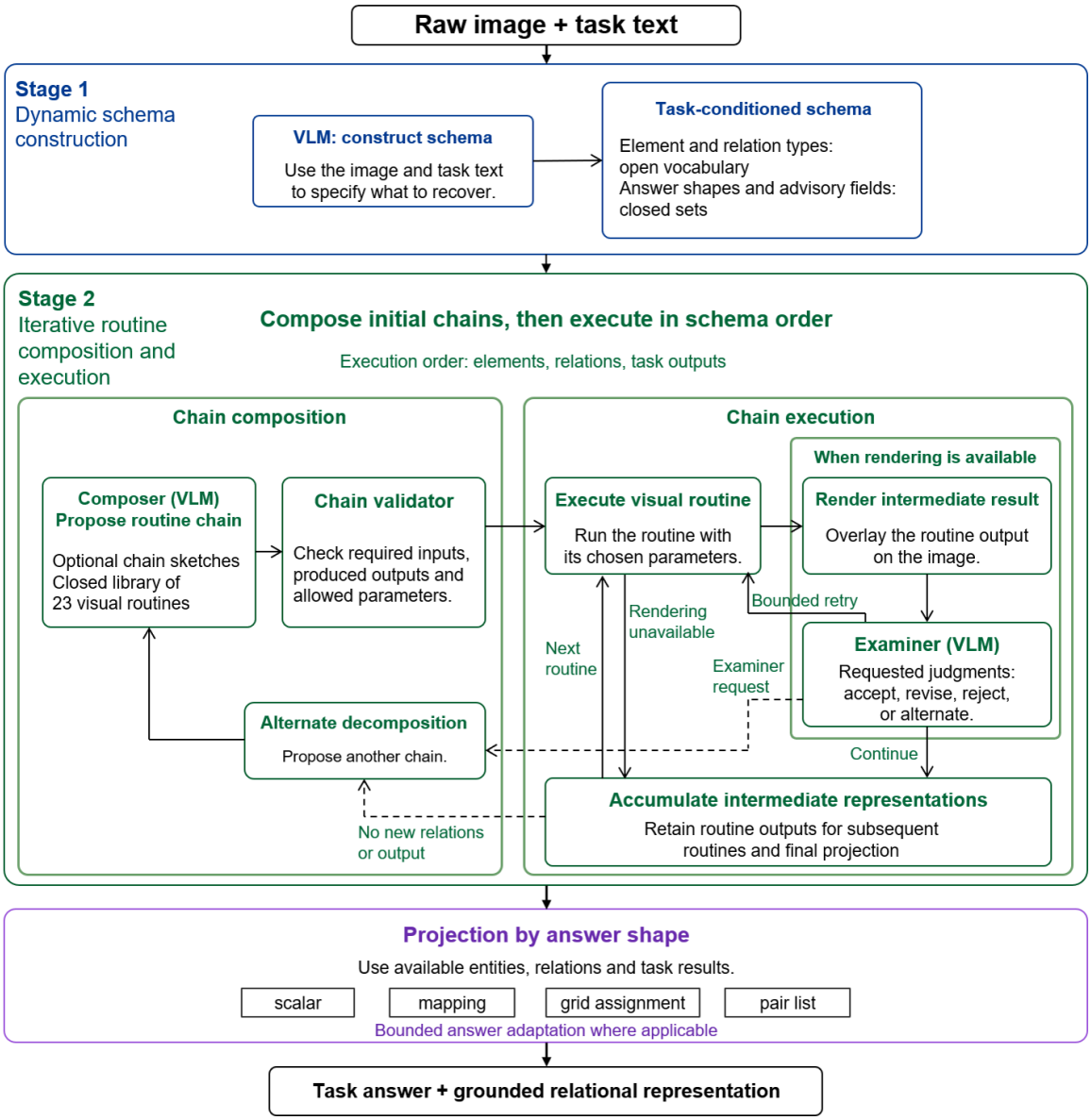


Figure 2. NVE workflow. A VLM constructs a task-conditioned schema, composes validated chains from the closed visual-routine library, and examines outputs where renderings are available. Routine results accumulate into a grounded representation that is projected into the required answer form under bounded adaptation. At completion of a relation or task-output chain, alternate decomposition can also be triggered when neither new relations nor a nonempty final output is produced.

Algorithm 1. NVE execution

Input: image I, task Q, routine catalog R, execution budget B

Output: accumulated representation G and available task answers A

1. Initialize the accumulated representation G and answer store A.
2. Infer and validate a task-conditioned schema S from I and Q.
3. Compose and validate the initial chains for S before execution. Order them as elements, structural relations, and task outputs.
4. For each planned chain, while the budget permits:
   a. Execute its routines in order, reusing available results where applicable. Apply eligible task checks and examine outputs where renderings are available. When the revision policy permits, revise the parameters and allow at most one routine retry.
   b. Accumulate available entities, relations, and task results.
   c. If the examiner requests an alternate, or a relation/task chain adds no relations and leaves no result in its final declared output, recompose with its failure history and repeat (a-b), allowing at most two alternates. A measured zero counts as a result. Stop on a repeated or empty chain or when the budget prevents execution.
5. Restore saved task-answer slots where applicable and project by the requested answer shape. Pair projection can add filtered relations to G. Grid adaptation applies a bounded global token map.
6. Return G and available answers, including partial output if a budget limit stopped execution.

3.2 Dynamic Schema Construction

Given the raw image and task text, the Stage-1 VLM makes one structured call to infer a schema for that item. The schema is constructed anew for each image and task rather than selected from a predefined diagram genre. It specifies element and relation types in the task's own vocabulary, task outputs, and advisory facts. It does not detect specific instances or choose executable routines. Element and relation names are open vocabulary, so terms such as organism, station, or consumption need not be hard-coded. The answer shape is drawn from the closed set: pair_list, scalar, mapping, or grid_assignment. Additional advisory fields such as answer value type, rendering class, and direction convention also use system-defined values. Open names guide later composition and persist as labels or predicates, but they never select executable code. Closed fields can be consumed mechanically. In particular, answer shape selects projection, while answer value type and direction convention constrain validation or bounded interpretation. The structured response is validated and can receive one repair request. Persistent failure yields an empty schema rather than a genre-specific fallback. Stage 2 uses the resulting schema as the semantic target for composition.

3.3 A Library of General, Compositional Visual Routines

NVE exposes a closed catalog of 23 visual routines from which the composer builds a perception process. A routine is a typed executable operation over a shared collection of intermediate representations. These include results used during execution, such as detected text, connectors, and sites (locations derived from detected structure), as well as the entities, relations, and answers retained for final output. Each routine declares the representations it requires and produces, together with its allowed parameters. Grounded outputs carry the image evidence and routine provenance needed by later routines and the final representation. Several routines wrap fixed off-the-shelf visual components, including UNITS (Kil et al., 2023) for scene-text spotting, LSD (Grompone

von Gioi et al., 2012) for line-segment detection, and SAM 3 (Carion et al., 2025) for segmentation, together with classical geometry and reusable structural operations. Most entries are primitives such as text detection, lattice detection, site localization, curve tracing, spatial relation tests, symbol reading, and answer projection. Three of the 23 visual routines are composite operations with the same typed interface but a fixed private sub-pipeline. For example, *extract_arrows* combines line detection, chaining, segmentation, route arbitration, arrowhead detection, and merging. The composer can choose the composite and its exposed parameters but cannot reorder its internal steps. Because executable chains are restricted to this catalog, the routines form the only path from pixels to perceptual structure.

Table 1. Visual routine catalog. Each routine declares required and produced representations and a typed parameter interface used during composition and validation.

Routine	**Description**
detect_text	Detects and recognizes scene text with limited scale, split, and merge refinement.
detect_strokes	Detects line segments and chains them into polylines.
detect_regions	Produces promptable region masks and reuses cached proposals.
detect_lattice	Detects rows, columns, and cell boxes from regular rulings.
select	Filters evidence by visual properties such as area, shape, color, or adjacency.
locate_sites	Enumerates sites from structure, such as cells, margins, and curve endpoints.
anchor_names	Binds detected text to positioned named handles.
group_elements	Groups evidence by color, collinearity, or connectivity.
trace_curve	Traces a colored curve from seed points.
find_crossings	Finds crossings between stroke groups.
relate_pair	Computes a closed-vocabulary spatial relation between two regions.
bind_nearest	Binds connector endpoints to nearby entities.
orient_connector	Assigns connector direction from measured head geometry, with VLM tie-breaking on abstention.
match_correspondence	Associates text labels with illustrations or masks.
read_glyph	Reads or classifies a symbol at a localized site using a schema-supplied vocabulary.
count_of	Counts items in an input collection.
report_relation	Returns a relation label from computed pair relations.
tabulate_lattice	Assembles per-site readings into a grid.
pairs_table	Builds pair lists or per-pair counts from extracted structure.
measure_structure	Reports dimensions or other structure measurements.
classify_text_roles	Combines OCR fragments into schema-typed entity labels.
extract_arrows	Extracts connectors using fixed line and segmentation routes plus head detection.
revise_relations	Verifies or drops extracted relations without proposing new ones.

The library favors decomposition when an intermediate result can be reused by other chains or when the order of steps depends on the task. For example, path counting is expressed through

separate routines for anchoring names, tracing curves, and assembling pair counts. Attention is also explicit. *select* filters existing evidence by properties, while *locate_sites* enumerates where later routines should operate from detected structure. These steps remain separate because their outputs have independent uses and can be examined before more expensive reading. In contrast, a fixed sequence with private intermediate representations is kept as a composite operation. The boundary can change as the library evolves. If an internal result later becomes useful to other tasks, that part can be promoted to a primitive.

3.4 Composition, Validation, and Chain Sketches

The composer role uses a VLM to author a chain over the routine catalog for each schema item. Each element-type chain can construct multiple instances. Chains share intermediate representations, and a chain may compute supporting structure beyond its target. Later chains can reuse available results, as illustrated in Appendix A. The composer's context includes the schema item, lightweight measured image facts, the routine catalog with parameter documentation and contracts, and optional chain sketches. On alternate decomposition it also receives the recent failed-chain history. Each routine contract specifies required representations, produced representations, and typed parameters with allowed values. The validator additionally checks that task chains produce the required answer slot with a compatible value type. Open schema names tell the composer what visual structure it is trying to construct, while executable steps and parameter values come only from the closed catalog.

The composer may also receive chain sketches as advisory examples. A chain sketch is not executable by itself. It is a worked routine sequence with typical parameters and a short "use when" description. For example, a sketch for counting crossings between selected curves suggests detecting strokes, selecting them by color, finding crossings, and then counting them. The composer may adopt, modify, reorder, or ignore a sketch, and validation still applies. Unlike a composite routine, a sketch does not hide a fixed executable sub-pipeline. The prompt deduplicates hand-seeded sketches that share the same routine-name sequence. Section 5.4 tests the entire sketch block.

Before execution, the chain validator simulates the proposed chain against routine contracts. It checks catalog membership, allowed parameters, availability of required inputs, and task-answer requirements. It first applies up to three mechanical fixes, such as inserting *detect_strokes* when a later step requires strokes or rewiring a uniquely resolvable input reference. These mechanical repairs were used on about 6% of composition calls in the full evaluation. If violations remain, the composer receives one feedback round naming the precise defects and re-authors the chain once. Persistent invalidity is recorded as composition failure rather than searched combinatorially. A relation or task chain that adds no relations and leaves no result in its final declared output can also trigger alternate decomposition. The composer receives the failed-chain history and may propose a different chain, at most twice. Section 5.1 tests this recovery mechanism.

3.5 The Visualize-Examine-Refine Loop

The examiner role provides bounded VLM guidance around routine execution. For routines with a renderer, NVE renders the output back onto the image as numbered boxes, connector overlays, or selected regions. The VLM judges the rendered output against the step's expected result and is asked to return one of four outcomes: accept, revise, reject, or request alternate composition, plus hints for adjusting parameters defined by the routine. For example, failed arrow extraction can be retried with a different extraction strategy suggested by the examiner. For steps that produce a count, NVE can also ask the VLM to check the routine-measured count in the terms of the original task. If the VLM agrees, the measured count can be accepted directly. If it disagrees, the conflict is recorded.

Refinement is deliberately bounded. A rejected or empty flagged result can receive at most one parameter-revised re-execution. A revise verdict alone does not trigger re-execution when the routine has produced a nonempty output. Missing or uncertain outputs may require further checking. A confirmed zero is treated as a valid result and does not by itself trigger recomposition. Section 5.2 tests the combined examination and checking layer.

3.6 Projection and Closure-Gated Answer Adaptation

Stage 2 ends with projection. The schema's closed answer-shape field selects one of four shape-generic projections: scalar, mapping, grid assignment, or pair list. Open schema names remain semantic labels and do not select computational branches. Some tasks require an answer convention that differs from the extracted representation, such as a particular grid-token vocabulary or relation orientation. NVE allows such re-expression through bounded transformations of the extracted structure.

A closure gate is a code-enforced restriction on final VLM transformations of structure that has already been extracted. For pair lists, this gate runs inside projection. The VLM sees the task, image, and extracted candidate pairs and may keep, reverse, or drop those pairs, but it cannot add a new pair. Invalid entries are filtered individually. An unparseable response or one retaining no valid pairs leaves the deterministic candidate list in place. Projected pairs are added to the accumulated representation without removing earlier harvested relations. For grids, a post-projection adapter asks the VLM only for a global map from extraction tokens to the task's answer tokens. Code applies the validated map mechanically to every cell, preserving cell positions but potentially merging token classes. An invalid grid map receives one retry and then uses a mechanical declared-vocabulary fallback when available. Otherwise the raw projected grid stands. Scalar and mapping outputs pass through unchanged. Thus adaptation can change answer convention through bounded transformations of extracted structure.

4. Evaluation and Analysis

4.1 Benchmarks and Metrics

FoodWebs is scored on directed consumption relations using exact matching after label normalization, reported as macro mean per-diagram F1. NVE is scored on task relations in the accumulated representation, including relations harvested before final pair-list adaptation. BlindTest uses normalized exact match. BlindTest Extended uses normalized exact match for scalar answers and per-key accuracy for subway pair tables. VGRP reports mean cell accuracy and exact-board transcription. All compared systems use the same task-answer normalization before scoring so that scores reflect task answers rather than response formatting.

4.2 Setup

Per-item budgets cap guide calls and wall-clock time: 120 calls/900 s for FoodWebs, 45 calls/600 s for VGRP, and 21 calls/600 s for BlindTest and BlindTest Extended. A ceiling stops further execution but keeps the partially constructed grounded representation, which is scored normally. Items are never dropped. Cached routine outputs and verification judgments are reused, and a small call reserve protects answer production from structural exploration. The pipeline receives no dataset metadata beyond image and task text, and every solve emits an audit trace of routine calls, outputs, examinations, and repairs. NVE has no trained parameters. Development consisted of explicit refinements to general-purpose visual routines and prompts. The evaluation sets were consulted to diagnose failures, but every resulting change was required to remain general-purpose. The implementation and reusable prompt scaffolding are explicitly checked to remain free of benchmark-specific vocabulary and logic.

The same-backbone comparison uses Qwen3.6-35B-A3B (Qwen Team, 2026) for both NVE and direct prompting at temperature 0. It is not decoding-matched: the NVE guide uses a fixed local thinking-off configuration, while direct Qwen uses its fixed baseline configuration with thinking on except for VGRP, where thinking frequently exhausted the output budget and produced empty responses. Thinking is therefore a configuration choice, not an experimental factor. Direct Gemini 3.7 Flash (Google DeepMind, 2026) is reported as a frontier reference. SAM 3, UNITS, LSD, and classical geometry are used off the shelf and held fixed (Canny, 1986; Carion et al., 2025; Galambos, Matas, & Kittler, 1999; Grompone von Gioi et al., 2012; Kil et al., 2023). NVE is training-free end to end. All guide sampling is at temperature 0. The reported full-evaluation numbers are single evaluations under a fixed serving setup. NVE and the same-backbone direct baseline use different serving APIs, with the decoding differences described above. For FoodWebs, the direct prompt explicitly specifies prey-to-predator direction, while NVE is asked to list the consumption relations shown by arrows.

4.3 Main Results

Table 2 shows that, relative to direct Qwen3.6, NVE is lower on FoodWebs by 0.247 macro F1 (0.503 vs. 0.750) and on BlindTest by 0.106 (0.783 vs. 0.889), but higher on BlindTest Extended

by 0.179 (0.754 vs. 0.575) and on VGRP cell accuracy by 0.169 (0.762 vs. 0.593). VGRP exact-board accuracy remains lower (0.336 vs. 0.378). The divergence between cell and exact-board accuracy indicates that NVE improves local cell perception without eliminating every residual cell error on a board. Across BlindTest and BlindTest Extended, NVE changes only 0.029, while direct Qwen changes 0.314 and direct Gemini 0.159. Because the suites are not item-paired and differ in family composition and task format, this is a cross-suite robustness trend, not a controlled difficulty curve. Direct Gemini remains the strongest absolute baseline on all four evaluations.

Table 2. Main results. FoodWebs reports macro F1 (NVE pooled micro F1 = 0.464). BlindTest reports normalized exact match on 470 items (NVE equal-task-weight accuracy = 0.744). BlindTest Extended reports mean accuracy on 120 generated items. VGRP reports cell / exact-board accuracy. For context, Rahmanzadehgervi et al. (2024) report 50.2% for GPT-4o and a 58.1% average across four models evaluated in that study.

System	Food-web macro F1	BlindTest	BlindTest Extended	VGRP cell / exact
Direct Gemini 3.7 Flash	0.881	0.957	0.799	0.929 / 0.879
Direct Qwen3.6	0.750	0.889	0.575	0.593 / 0.378
NVE (Qwen3.6 guide)	0.503	0.783	0.754	0.762 / 0.336

4.4 Generality and Reuse

A catalog organized around benchmark-specific pipelines would tend to partition by evaluation set. Cross-genre reuse instead indicates that routines are being recomposed across tasks. The reuse matrix in Table 3 measures deployment, not necessity. All 23 routines execute somewhere, 22 execute on at least two evaluation sets, and 20 execute on at least 5% of items in at least two sets. Text/connector routines span both BlindTest suites and FoodWebs, while lattice/glyph routines are near-universal on VGRP and reused by BlindTest grid tasks. *pairs_table* executes on all four sets (0.13, 0.17, 1.00, 0.25). Reuse between BlindTest and BlindTest Extended is weaker evidence because they share task families. The stronger cross-genre result is that the same closed catalog also supports FoodWebs and VGRP.

Table 3. Fraction of items on which each routine executed in the reported full evaluation. A value of 0 means no execution. <.01 denotes a nonzero rate below 0.005, with the raw item count in parentheses.

Routine	BlindTest (470)	BlindTest Extended (120)	FoodWebs	VGRP
anchor_names	0.11	0.15	<.01 (1)	0.01
bind_nearest	0.21	0.23	1.00	0.01
classify_text_roles	0.31	0.30	1.00	0.02
count_of	0.69	0.57	0	0.25
detect_lattice	0.19	0.17	0	0.99

Routine	BlindTest (470)	BlindTest Extended (120)	FoodWebs	VGRP
detect_regions	0.66	0.49	0.02	0.21
detect_strokes	0.23	0.51	0.02	0.051
detect_text	0.31	0.31	1.00	0.02
extract_arrows	0.21	0.23	1.00	0.01
find_crossings	0.08	0.17	0	<.01 (2)
group_elements	0.11	0.23	0	0.23
locate_sites	0.41	0.33	0.01	0.99
match_correspondence	0.16	0.13	0.70	<.01 (4)
measure_structure	0.19	0.17	0	0.03
orient_connector	0.04	0.01	1.00	<.01 (1)
pairs_table	0.13	0.17	1.00	0.25
read_glyph	0.39	0.33	0	0.99
relate_pair	0.41	0.23	0	0.13
report_relation	0.29	0.02	0	0.02
revise_relations	0	0	0.30	0
select	0.67	0.54	0.01	0.25
tabulate_lattice	0.18	0.17	0	0.97
trace_curve	0.17	0.33	<.01 (1)	0.044

4.5 Error Attribution as an Architectural Property

A central benefit of NVE's explicit intermediate representation is error attribution: intermediate results can be traced through the composed chain, routine evidence, and bounded arbitration or adaptation. Comparing all 6,134 FoodWeb gold edges with the retained entities and relations gives a diagnostic breakdown, with missed edges on budget-limited items grouped separately. Of the gold edges, 53.8% are recovered with correct orientation, 16.4% have an endpoint absent from the retained entities, 15.9% have both endpoints but no retained pair in either direction, 8.3% have a retained pair but no correctly oriented scored relation, and 5.5% are missed on the 31 items that reached a budget ceiling. Missing entities and pairs are the largest diagnostic categories, which the grounded representation makes directly inspectable.

5. Ablation Studies

We evaluate four architectural interventions with item-level paired evaluations. The main ablation evaluation uses a fixed, seeded sample of 190 BlindTest items, 80 BlindTest Extended items, 120 FoodWebs, and 228 VGRP boards. Each altered configuration is compared with the full system on identical items under the same serving setup. The full-system subset scores closely track the

corresponding full-evaluation levels. We report the paired change from the corresponding full-system score for each intervention.

Table 4. Item-level paired ablations on the fixed 618-item evaluation. Each cell reports the configuration score, with its change from the paired full-system configuration in parentheses.

Configuration	BlindTest	BlindTest Extended	FoodWebs	VGRP cell
Full system	0.779	0.735	0.491	0.785
No alternate decomposition	0.753 (-0.026)	0.727 (-0.008)	0.487 (-0.003)	0.786 (+0.001)
No examination/checking layer	0.774 (-0.005)	0.663 (-0.073)	0.477 (-0.014)	0.781 (-0.004)
Gemini 3.7 Flash guide	0.821 (+0.042)	0.679 (-0.056)	0.495 (+0.004)	0.798 (+0.013)
No chain sketches	0.637 (-0.142)	0.598 (-0.138)	0.477 (-0.014)	0.773 (-0.012)

5.1 Alternate Decomposition

Removing alternate decomposition has its largest aggregate effect on BlindTest, a change of -0.026 in Table 4, concentrated in circled-letter and Olympic-counting items. On these sampled items, recomposition appears to be a modest recovery mechanism when the first composition fails rather than a broadly load-bearing component.

5.2 Examine-Refine Loop

Disabling all per-step examinations also removes checks of uncertain outputs and the count check described in Section 3.5. The largest effect is on BlindTest Extended, a change of -0.073 in Table 4, with losses concentrated in shape-counting and circled-letter tasks. The intervention saves 7% to 21% of guide calls by benchmark, about 11% item-weighted. The combined examination and checking layer therefore shows a localized benefit within BlindTest Extended.

5.3 Guide Exchangeability

Replacing Qwen3.6 with Gemini 3.7 Flash produces mixed changes of at most 0.056 across the four evaluations (Table 4). These results indicate that NVE can use either tested VLM as its guide within the same architecture. The Gemini guide ran at temperature 0 with default dynamic thinking and no thinking budget set, whereas Qwen runs thinking-off, so the swap is not decoding-matched. This comparison concerns the two NVE guide configurations rather than the direct-Gemini baseline in Table 2.

5.4 Chain Sketches

Removing chain sketches produces the largest effects in the ablation set: -0.142 on BlindTest and -0.138 on BlindTest Extended (Table 4). On BlindTest, circled-letter accuracy drops from 0.69 to 0.12 on 42 sampled items and Olympic counting drops by about 0.20, while touching, nested-square, counting-grid, and line-intersection families remain flat. The result distinguishes routing priors from validator repair: sketches help determine what chain to try, whereas validation makes a proposed chain contract-valid. Overall, the clearest measured control value lies in composition-side routing, especially chain sketches. Alternate decomposition provides a smaller observed recovery effect, while the combined examination/checking layer has a localized effect within BlindTest Extended.

6. Conclusion and Future Work

NVE presents a training-free, interpretable mechanism for structural diagram perception in which a VLM guides composition over a closed library of grounded visual routines. The architecture separates open semantic specification from grounded perceptual construction and mechanically constrains final adaptation to bounded transformations of extracted structure. On the full evaluation, NVE with Qwen3.6 outperforms direct Qwen3.6 on BlindTest Extended (0.754 versus 0.575) and VGRP cell accuracy (0.762 versus 0.593), while trailing it on BlindTest (0.783 versus 0.889), food-web macro F1 (0.503 versus 0.750), and VGRP exact-board accuracy (0.336 versus 0.378). Paired ablations identify chain sketches as the strongest measured control component on the two BlindTest suites, while removing the combined examination/checking layer has a smaller localized effect on BlindTest Extended.

The current system has clear boundaries, and they set the agenda. Its recall depends on the coverage of off-the-shelf visual components, and the examiner can only judge evidence made visible by the system's renderings. Chain sketches remain a hand-seeded prior until the system can accumulate them from successful solves. The grounded relational outputs are designed for CogSketch compatibility. Connecting them to analogical reasoning over constraint and planning fragments is a natural next step.

Direct Gemini 3.7 Flash remains the strongest direct baseline across all four evaluations, reaching 0.881 food-web macro F1, 0.957 on BlindTest, 0.799 on BlindTest Extended, and 0.929 cell accuracy with 0.879 exact-board accuracy on VGRP. This is consistent with an important advantage of strong general-purpose VLMs: broad visual coverage and robust holistic interpretation without requiring every useful perceptual operation to be anticipated in a fixed routine library. NVE's intended advantage is different: grounding, inspectability, explicit relational structure, and the ability to attribute errors to particular computations. Its perceptual coverage remains bounded by the capabilities and compositions available in the routine library.

Another promising future direction is therefore a more flexible form of neuro-symbolic visual understanding that focuses on structural representations in higher-level reasoning instead of those in perception: VLMs directly invoke visual tools, use their grounded outputs to propose an explicit structural representation, inspect that representation against the image and task, and revise it

iteratively. Such a design could combine the broader, more holistic visual competence of frontier VLMs with the explicit grounding and revisability that motivate NVE, while providing better flexibility than the perceptual construction through a fixed composition of predefined routines.

Acknowledgements

This research was supported by the Machine Learning, Reasoning, and Intelligence Program of the US Office of Naval Research, Grant N00014-23-1-2294.

References

Biederman, I. (1987). Recognition-by-components: A theory of human image understanding. *Psychological Review, 94*(2), 115–147.

Canny, J. F. (1986). A computational approach to edge detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence, PAMI-8*(6), 679–698.

Carion, N., Gustafson, L., Hu, Y.-T., Debnath, S., Hu, R., Suris, D., Ryali, C., Alwala, K. V., Khedr, H., Huang, A., Lei, J., Ma, T., Guo, B., Kalla, A., Marks, M., Greer, J., Wang, M., Sun, P., Rädle, R., Afouras, T., Mavroudi, E., Xu, K., Wu, T.-H., Zhou, Y., Momeni, L., Hazra, R., Ding, S., Vaze, S., Porcher, F., Li, F., Li, S., Kamath, A., Cheng, H. K., Dollár, P., Ravi, N., Saenko, K., Zhang, P., & Feichtenhofer, C. (2025). SAM 3: Segment anything with concepts. arXiv preprint arXiv:2511.16719.

Chen, K., & Forbus, K. (2021). Visual relation detection using hybrid analogical learning. In *Proceedings of the Thirty-Fifth AAAI Conference on Artificial Intelligence (AAAI-21)* (pp. 801–808).

Chen, K., Forbus, K., Srinivasan, B. V., Chhaya, N., & Usher, M. (2023). Sketch recognition via part-based hierarchical analogical learning. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence (IJCAI-23)* (pp. 2967–2974).

Chen, Z., Sun, R., Liu, W., Hong, Y., & Gan, C. (2024). GENOME: Generative neuro-symbolic visual reasoning by growing and reusing modules. In *International Conference on Learning Representations (ICLR 2024)*.

Cohn, A. G., & Renz, J. (2008). Qualitative spatial representation and reasoning. In F. van Harmelen, V. Lifschitz, & B. Porter (Eds.), *Handbook of Knowledge Representation* (pp. 551–596). Elsevier.

Forbus, K. D., Chen, K., Xu, W., & Usher, M. (2024). Hybrid primal sketch: Combining analogy, qualitative representations, and computer vision for scene understanding. *Advances in Cognitive Systems, 11*, 1–16.

Forbus, K. D., & Lovett, A. (2021). Same/different in visual reasoning. *Current Opinion in Behavioral Sciences*, 37, 63–68. https://doi.org/10.1016/j.cobeha.2020.09.008.

Forbus, K. D., Usher, J., Lovett, A., Lockwood, K., & Wetzel, J. (2011). CogSketch: Sketch understanding for cognitive science research and for education. *Topics in Cognitive Science, 3*(4), 648–666.

Galambos, C., Matas, J., & Kittler, J. (1999). Progressive probabilistic Hough transform for line detection. In *Proceedings of the 1999 IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, Vol. 1 (pp. 554–560).

Google DeepMind. (2026). Gemini 3.7 Flash: Model card. Google DeepMind.

Grompone von Gioi, R., Jakubowicz, J., Morel, J.-M., & Randall, G. (2012). LSD: A line segment detector. *Image Processing On Line, 2*, 35–55.

Gupta, T., & Kembhavi, A. (2023). Visual programming: Compositional visual reasoning without training. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 14953–14962).

Hu, Y., Shi, W., Fu, X., Roth, D., Ostendorf, M., Zettlemoyer, L., Smith, N. A., & Krishna, R. (2024). Visual Sketchpad: Sketching as a visual chain of thought for multimodal language models. *Advances in Neural Information Processing Systems, 37,* 139348–139379.

Kil, T., Kim, S., Seo, S., Kim, Y., & Kim, D. (2023). Towards unified scene text spotting based on sequence generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 15223–15232).

Krishnamurthy, J., Tafjord, O., & Kembhavi, A. (2016). Semantic parsing to probabilistic programs for situated question answering. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing* (pp. 160–170). Association for Computational Linguistics.

Lu, P., Peng, B., Cheng, H., Galley, M., Chang, K.-W., Wu, Y. N., Zhu, S.-C., & Gao, J. (2023). Chameleon: Plug-and-play compositional reasoning with large language models. *Advances in Neural Information Processing Systems, 36*, 43447–43478.

Marr, D. (1982). *Vision: A Computational Investigation into the Human Representation and Processing of Visual Information*. W. H. Freeman.

Qwen Team. (2026). Qwen3.6-35B-A3B: Agentic coding power, now open to all. Qwen.

Rahmanzadehgervi, P., Bolton, L., Taesiri, M. R., & Nguyen, A. T. (2024). Vision language models are blind. In *Proceedings of the Asian Conference on Computer Vision (ACCV)* (pp. 18–34).

Ren, Y., Tertikas, K., Maiti, S., Han, J., Zhang, T., Süsstrunk, S., & Kokkinos, F. (2025). *VGRP-Bench: Visual grid reasoning puzzle benchmark for large vision-language models*. arXiv preprint arXiv:2503.23064.

Surís, D., Menon, S., & Vondrick, C. (2023). ViperGPT: Visual inference via Python execution for reasoning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)* (pp. 11888–11898).

Ullman, S. (1984). Visual routines. *Cognition, 18*, 97–159.

Xu, W., & Forbus, K. D. (2025). Understanding diagrams with explicit intermediate visual representation. In *Proceedings of the Twelfth Annual Conference on Advances in Cognitive Systems (ACS-2025)* (pp. 224–240).

Yang, J., Zhang, H., Li, F., Zou, X., Li, C., & Gao, J. (2023a). *Set-of-Mark prompting unleashes extraordinary visual grounding in GPT-4V*. arXiv preprint arXiv:2310.11441.

Yang, Z., Li, L., Wang, J., Lin, K., Azarnasab, E., Ahmed, F., Liu, Z., Liu, C., Zeng, M., & Wang, L. (2023b). *MM-REACT: Prompting ChatGPT for multimodal reasoning and action*. arXiv preprint arXiv:2303.11381.

Appendix A. Worked Examples and Intermediate Representations

These two evaluated examples follow NVE from an input image and requested schema to a projected answer. FoodWeb 1843 illustrates entity grounding, connector extraction, and a retry following rendered examination. Odd-Even Sudoku item 45 illustrates cell geometry, digit and shading representations, shared results across chains, and grid transcription. Each example presents its schema, explains the chains composed for its element types, relation types, and task outputs, and follows execution with structured output excerpts. Examination is shown alongside the relevant visual result. The schema specifies the requested structure, while the execution and final output show what was produced.

The examples connect the staged construction discussed in Section 2.1 to the explicit representations sought by CogSketch in Section 2.2. An interface could be engineered to automatically convert NVE's structured intermediate representations into predicate descriptions for CogSketch, preserving identifiers, attributes, relations, and available image groundings. The accompanying predicate expressions illustrate this mapping using an explanatory vocabulary, including names prefixed with nve. The evaluated runs produce the intermediate representations but do not perform the conversion. Image coordinates are given in pixels.

A.1 FoodWeb 1843

The task is to recover directed consumption relations, following the drawn arrows.

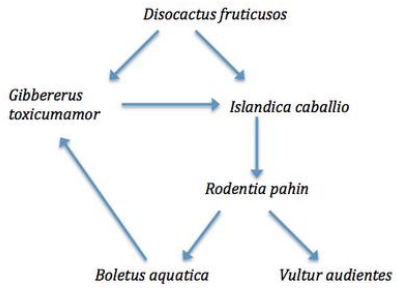


Figure A1. Input diagram.

Schema

```
{
 "elements": [{"name": "Organism"}],
 "relations": [{"name": "consumes", "args": ["Organism", "Organism"]}],
 "task_outputs": [{"name": "consumption_relations", "shape": "pair_list"}]
}
```

A.1.1 Routine Composition

The composer constructs three chains, targeting the Organism element type, the consumes relation type, and the consumption_relations task output. They execute in that order over shared intermediate representations. All three use the routine sequence below. The Organism chain constructs the organisms collectively, then continues through arrow and pair extraction. Its target does not restrict it to entity construction.
detect_text → classify_text_roles → extract_arrows → bind_nearest → orient_connector → pairs_table

A.1.2 Organism Chain Execution

Step 1. *detect_text*

From the original image, *detect_text* returns six text fragments containing organism labels.

Output excerpt	**Possible predicates**
{"text": "Islandica caballio"}	(nveText text_2 "Islandica caballio")

Step 2. *classify_text_roles*

Using the text fragments and image, *classify_text_roles* constructs six labeled entities. Each box is an approximate image anchor for a label.

Output excerpt	**Possible predicates**
{"id": "entity_2", "label": "Islandica caballio", "bbox": [265.5625, 96.5625, 305.4375, 136.4375]}	(nveLabel entity_2 "Islandica caballio") (nveAnchorBox entity_2 image_1843 265.5625 96.5625 305.4375 136.4375)

Rendered examination requests revised labeling. The policy retains the nonempty output without retrying this step.

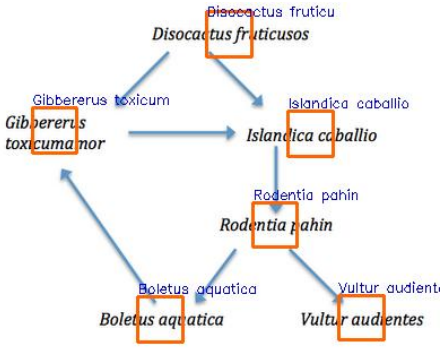


Figure A2. Entity anchors shown to the examiner.

Step 3. ***extract_arrows***

Arrow extraction from the original image initially returns no arrows. The examiner rejects this output, triggering a bounded retry that returns seven arrows and is accepted.

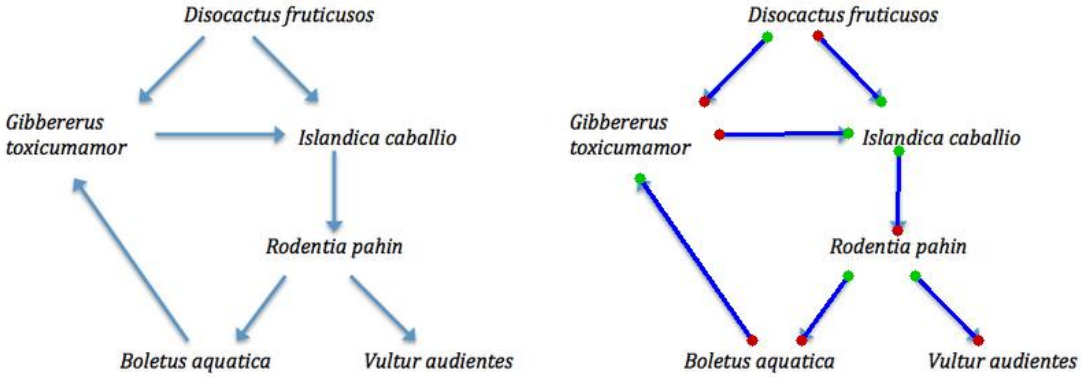


Figure A3. Initial extraction (left) and successful retry (right).

Output excerpt

{"id": "arrow_5",
"tail_xy": [255.0, 127.0],
"head_xy": [254.0, 185.0]}

Possible predicates

(nveTail arrow_5 image_1843
255 127)
(nveHead arrow_5 image_1843
254 185)

Step 4. ***bind_nearest***

Associate arrow endpoints with entity anchors, producing seven candidate pairs.

Step 5. ***orient_connector***

Assign subject and object from arrow geometry. All seven directions in this chain are determined geometrically.

Step 6. ***pairs_table***

Assemble the seven directed pairs for the chain output. Their entity identifiers connect the relations to the image anchors.

A.1.3 Relation and Task-Output Chains and Projection

The consumes and consumption_relations chains execute next. Both reuse detected text and repeat arrow extraction. Although their routine names match the Organism chain, they are separate executions targeting the relation and task output. Pair outputs from the Organism chain remain intermediate results rather than being recorded as Organism relations. The later chains collect relations under their respective predicates. Projection produces seven directed pairs, all retained by final adaptation.

Final directed pairs

```
[
  ["entity_1", "entity_2"],
  ["entity_0", "entity_1"],
  ["entity_3", "entity_5"],
  ["entity_3", "entity_4"],
  ["entity_4", "entity_1"],
  ["entity_2", "entity_3"],
  ["entity_0", "entity_2"]
]
```

Entity labels

```
entity_0: Disocactus fruticusos
entity_1: Gibbererus toxicumamor
entity_2: Islandica caballio
entity_3: Rodentia pahin
entity_4: Boletus aquatica
entity_5: Vultur audientes
```

Output excerpt

```
{"id": "con_5",
 "predicate": "consumption_relations",
 "args": ["entity_2", "entity_3"],
 "grounding": {
   "provenance": ["consum_5"]}}
```

Possible predicates

```
(consumption_relations
  entity_2 entity_3)
(nveDerivedFrom con_5 consum_5)
```

The relation arguments reuse entity IDs. A conversion can preserve entity labels and anchors, connector geometry, and relation provenance as connected facts.

A.2 Odd-Even Sudoku Item 45

The reference answer records visible digits, empty cells, and cell shading.

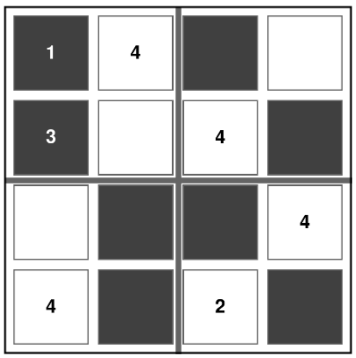


Figure A4. Input puzzle.

Schema

```
{
  "elements": [{"name": "grid_cell"}, {"name": "block"}],
  "relations": [
    {"name": "contains_value", "args": ["grid_cell", "value"]},
    {"name": "has_shading", "args": ["grid_cell", "shading_type"]},
    {"name": "belongs_to_block", "args": ["grid_cell", "block"]}
  ],
  "task_outputs": [{"name": "filled_cells", "shape": "grid_assignment"}]
}
```

A.2.1 Routine Composition

The composer constructs six chains, one for each of the two element types, three relation types, and one task output in the schema. They execute in that order over shared intermediate representations. The grid_cell and block chains have identical routines and parameters. The contains_value and has_shading chains share the four routine names below, but use different reading parameters. Here, belongs_to_block requests cell membership in a 2×2 subgrid, and filled_cells names the requested grid assignment.
detect_lattice → locate_sites → read_glyph → tabulate_lattice

A.2.2 Routine Execution

Element chains

Step 1. *detect_lattice*

Detect sixteen indexed cells from the original image. Cell positions are grounded in image boxes. Grid indices start at zero.

Output excerpt

```
{"id": "cell_0_0",
 "type_name": "cell",
 "attrs": {"row": 0, "col": 0},
 "grounding": {
   "bbox": [20, 20, 200, 200]}}
```

Possible predicates

```
(nveGridPosition cell_0_0 0 0)
(nveImageBox cell_0_0
  image_oddeven_45
  20 20 200 200)
```

Step 2. *locate_sites*

Create sixteen cell-interior reading sites from the detected geometry. Digit and shading routines use these localized image regions.

Step 3. *read_glyph*

Read seven digits from the image at the cell-interior sites using the OCR channel.

Output excerpt

```
{"text": "1",
 "polygon": [[111, 103], [128, 103],
          [128, 135], [111, 135]]}
```

Possible predicates

```
(nveText text_0 "1")
(nveImagePolygon text_0
  image_oddeven_45
  ((111 103) (128 103)
   (128 135) (111 135)))
```

Step 4. *tabulate_lattice*

Associate the digit readings with their grid positions, leaving other positions empty. This digit grid remains available for final projection.

Output excerpt

```
{"grid": [
 ["1", "4", "*", "*"],
 ["3", "*", "4", "*"],
 ["*", "*", "*", "4"],
 ["4", "*", "2", "*"]
]}
```

Possible predicates

```
(nveCellValue cell_0_0 "1")
(nveCellValue cell_0_1 "4")
(nveCellValue cell_1_0 "3")
(nveCellValue cell_3_2 "2")
```

Steps 1–4 execute the grid_cell chain. The block chain is skipped because its routines and parameters exactly duplicate that chain.

Relation chains

The contains_value chain reuses the lattice and sites, then reads and tabulates digits again. The has_shading chain reuses the same geometry but reads shading, as shown next.

Step 5. *read_glyph*

The has_shading chain reuses cell geometry and reading sites. Shading reading classifies dark cells as true and light cells as false.

Step 6. *tabulate_lattice*

Tabulate shading readings by grid position. The saved key filled_cells denotes this Boolean shading grid, with eight dark cells. The digit and shading representations share the same grounded cell identifiers.

Output excerpt

```
{"filled_cells": [
 [true, false, true, false],
 [true, false, false, true],
 [false, true, true, false],
 [false, true, false, true]
]}
```

Possible predicates

```
(nveDarkCell cell_0_0 true)
(nveDarkCell cell_0_1 false)
(nveDarkCell cell_0_2 true)
(nveDarkCell cell_0_3 false)
```

The schema's *belongs_to_block* relation requests identifying which 2×2 subgrid contains each cell. The composed chain instead reads the digits inside the cells and returns no cell-to-subgrid assignments, failing to produce the requested relation.

Task-output chain

The schema requests a grid assignment named *filled_cells*. The composed chain instead counts region proposals and returns 12, failing to produce the requested grid. Validation inserts *detect_regions* and *select* to supply the count's inputs. The chain reuses cell geometry and sites, then reads shading before region detection.

detect_lattice → *locate_sites* → *read_glyph*
→ *detect_regions* → *select* → *count_of*

Step 7. *detect_regions*

Detect twelve region proposals from the original image.

Step 8. ***select***

Select all twelve region proposals. The examiner accepts the rendered selection, and execution continues to counting.

{"selection_id": "selection_0",
 "source": "region_proposals",
 "n_candidates": 12,
 "dropped_bboxes": []}

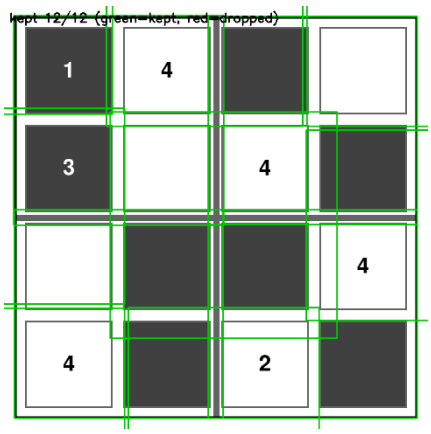


Figure A5. Selected region proposals shown to the examiner.

Step 9. ***count_of***

Count the selected region proposals, returning 12. This is the proposal count, rather than the number of digits or dark cells.

A.2.3 Final Projection

The chain targeting *filled_cells* returns a region count instead of the requested grid. Final projection uses the *grid_assignment* shape to select the digit grid retained from step 4 as answers.grid. The shading grid produced in step 6 remains available as answers.filled_cells, with true for dark cells and false for light cells. These final outputs therefore use previously computed representations despite the task-output chain's failure. Here, * marks an empty cell. Both displayed grids match all 16 reference positions.

Final digit grid

```
{"grid": [
  ["1", "4", "*", "*"],
  ["3", "*", "4", "*"],
  ["*", "*", "*", "4"],
  ["4", "*", "2", "*"]
]}
```

Final shading grid

```
{"filled_cells": [
  [true, false, true, false],
  [true, false, false, true],
  [false, true, true, false],
  [false, true, false, true]
]}
```

```
(nveCellValue cell_0_0 "1")
(nveDarkCell cell_0_0 true)
(nveImageBox cell_0_0
  image_oddeven_45
  20 20 200 200)
```

Shared cell identifiers connect digit and shading facts to image grounding. The final representation contains sixteen grounded cells without explicit contains_value, has_shading, or belongs_to_block relation instances.